

ПРАВИТЕЛЬСТВО САНКТ-ПЕТЕРБУРГА
КОМИТЕТ ПО НАУКЕ И ВЫСШЕЙ ШКОЛЕ

САНКТ-ПЕТЕРБУРГСКОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ
ПРОФЕССИОНАЛЬНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ
«АКАДЕМИЯ ТРАНСПОРТНЫХ ТЕХНОЛОГИЙ»
ИМЕНИ ГЕРОЯ СОЦИАЛИСТИЧЕСКОГО ТРУДА И.Г. ЗУБКОВА»

КУРСОВОЙ ПРОЕКТ

Специальность 09.02.07 Информационные системы и
программирование

МДК.07.01 Управление и автоматизация баз данных

Тема «Проектирование базы данных и разработка клиентского
приложения для автоматизации логистики курьерской доставки
малогабаритных грузов»

Пояснительная записка

АТТ2. КП0326. 000 ПЗ

Студента группы КИ-31 Власова Всеволода Анатольевича

ПРАВИТЕЛЬСТВО САНКТ-ПЕТЕРБУРГА
КОМИТЕТ ПО НАУКЕ И ВЫСШЕЙ ШКОЛЕ

САНКТ-ПЕТЕРБУРГСКОЕ ГОСУДАРСТВЕННОЕ
БЮДЖЕТНОЕ ПРОФЕССИОНАЛЬНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ
«АКАДЕМИЯ ТРАНСПОРТНЫХ ТЕХНОЛОГИЙ»

УТВЕРЖДАЮ
Зам. директора по учебной работе
_____/ Вишневская М.В./
«__» _____ 2026_ г.

ЗАДАНИЕ
на выполнение курсового проекта

1. Студент **Власов Всеволод Анатольевич**
Группа **КИ-31**
Специальность: 09.02.07 Информационные системы и программирование (базовый уровень)
Дисциплина/Междисциплинарный курс: МДК 07.01 «Управление и автоматизация баз данных»
2. Курсовой проект на тему **«Проектирование базы данных и разработка клиентского приложения для автоматизации логистики курьерской доставки малогабаритных грузов»**
утвержден приказом № 149/91к от «28» января 2026 г.
3. Структура курсового проекта включает следующие элементы:
 - пояснительная записка (титульный лист, задание, содержание, введение, теоретическая/основная часть, заключение, ссылки на используемую литературу, литература).

СОДЕРЖАНИЕ ПОЯСНИТЕЛЬНОЙ ЗАПИСКИ

Введение

1 Анализ требований и постановка задачи

- 1.1 Анализ предметной области.
- 1.2 Разработка UML диаграмм.
- 1.3 Разработка макета приложения.

2 Проектирование и разработка базы данных

- 2.1 Разработка ER-диаграммы на основе сырых данных.
- 2.2 Разработка физической модели базы данных в MS SQL Server.
- 2.3 Разработка и именование связей и ограничений.

3 Проектирование и разработка приложения

- 3.1 Разработка прототипа приложения.
- 3.2 Разработка описания проекта в GitHub.

Заключение

Литература

Рассмотрено на заседании ЦК № 5 «Информационные технологии»
протокол № 5 от «10» декабря 2025 года

Дата выдачи задания «__» ____ 2026 года

Дата сдачи выполненного проекта «__» _____ 2026 года

Руководитель КП _____ / Кошкин В.А. /

Подпись студента _____ / Власов В. А. /

Содержание

Введение	4
1 Анализ требований и постановка задачи	6
1.1 Анализ предметной области	6
1.2 Разработка UML-диаграмм	9
1.3 Разработка макета приложения	14
2 Проектирование и разработка базы данных	16
2.1 Разработка ER-диаграммы на основе сырых данных	16
2.2 Разработка физической модели базы данных в MS SQL Server	19
2.3 Разработка и именование связей и ограничений	23
3 Проектирование и разработка приложения	27
3.1 Разработка прототипа приложения	27
3.2 Разработка описания проекта в GitHub	30
Заключение	33
Литература	35

					АТТ2. КП0326. 000 ПЗ			
Изм.	Лист	№ докум.	Подп.	Дата				
Разраб.		Власов В.А.			Курсовой проект Пояснительная записка		Лит.	Лист
Пров.		Кошкин В.А.						Листов
.		.					3	35
							Группа КИ-31	

Введение

Современный рынок курьерской доставки демонстрирует устойчивый рост: по данным отраслевых аналитиков, объём отправок малогабаритных грузов в России ежегодно увеличивается на 15–20 %. Вместе с ростом числа заказов возрастает и нагрузка на операционные процессы: приём обращений, назначение курьеров, контроль исполнения, учёт финансов. Небольшие логистические компании нередко ведут эти процессы вручную — с помощью электронных таблиц, мессенджеров и телефонных звонков. Такой подход порождает типичные проблемы: потеря информации о заказах, задержки при назначении курьеров, отсутствие единой картины статусов, невозможность быстро сформировать отчёт для руководства.

Автоматизация посредством специализированной информационной системы позволяет устранить перечисленные недостатки: централизовать хранение данных, ввести чётко регламентированный жизненный цикл заказа, разграничить права доступа по ролям и обеспечить оперативную отчётность.

Цель данного курсового проекта — спроектировать реляционную базу данных и разработать клиентское приложение для автоматизации основных бизнес-процессов курьерской доставки малогабаритных грузов.

Для достижения поставленной цели необходимо решить следующие задачи:

- провести анализ предметной области и выявить ключевые бизнес-процессы;
- разработать UML-диаграммы (вариантов использования и последовательности), отражающие требования к системе;
- спроектировать концептуальную (ER) и физическую модели базы данных в MS SQL Server;
- реализовать клиентское WPF-приложение с ролевым доступом;

- разместить исходный код в публичном репозитории GitHub и обеспечить его документирование;
- провести функциональное тестирование системы в рамках учебного проекта.

Практическая значимость работы состоит в том, что разработанный программный продукт может служить основой для реальной системы управления доставкой небольшой курьерской службы.

					АТТ2. КП0326. 000 ПЗ	Лист
						5
Изм.	Лист	№ докум.	Подп.	Дата		

1 Анализ требований и постановка задачи

1.1 Анализ предметной области

Предметная область проекта — оперативная логистика курьерской доставки малогабаритных грузов. Под малогабаритным грузом понимается отправление, которое один курьер способен доставить лично (посылки, документы, небольшие товары интернет-магазинов).

Жизненный цикл заказа включает следующие последовательные этапы:

- создание заказа. Клиент формирует заявку на доставку, указывая адрес отправителя и получателя, тип груза, желаемый временной интервал и контактные данные сторон. Системе автоматически присваивается уникальный трек-номер;

- назначение курьера. Диспетчер просматривает новые заявки и назначает подходящего свободного курьера с учётом зоны доставки и текущей загруженности. Назначение может выполняться как вручную, так и автоматически — с помощью соответствующей хранимой процедуры;

- принятие заказа курьером. После уведомления курьер подтверждает готовность выполнить доставку; статус заказа переходит в состояние «Принят»;

- транспортировка груза. Курьер забирает отправление у отправителя, статус меняется на «В пути». По прибытии к получателю фиксируется промежуточное состояние «У получателя»;

- подтверждение доставки. Факт передачи груза получателю фиксируется в системе: сохраняются дата, время, при необходимости — подпись или код подтверждения. Статус заказа принимает конечное значение «Доставлен»;

- отмена заказа. На любом этапе до фактической передачи груза заказ может быть отменён диспетчером или клиентом с фиксацией причины;

					АТТ2. КП0326. 000 ПЗ	Лист
						6
Изм.	Лист	№ докум.	Подп.	Дата		

В таблице 1 представлены все возможные статусы заказа с их описанием.

Таблица 1 — Статусы заказа в системе

Статус	Описание
New	Заказ создан, курьер не назначен
Assigned	Курьер назначен, ожидает принятия
Taken	Курьер принял заказ
InTransit	Груз в пути (курьер выехал к отправителю)
AtRecipient	Курьер прибыл к получателю
Delivered	Доставка подтверждена
Cancelled	Заказ отменён

Помимо цикла обработки одного заказа, в системе реализованы следующие бизнес-процессы: ведение справочников (зоны доставки, типы грузов, тарифы), управление маршрутными листами (объединение нескольких заказов в один рейс курьера), формирование уведомлений о смене статуса, построение отчётов об эффективности курьеров.

Система предусматривает четыре роли с различными правами доступа. Перечень ролей и соответствующих функций приведён в таблице 2.

Таблица 2 — Роли пользователей системы

Роль	Основные функции
Клиент	Создание и просмотр своих заказов, отслеживание статуса по трек-номеру, получение уведомлений
Курьер	Просмотр назначенных заказов, изменение статусов в процессе выполнения, подтверждение доставки

Продолжение таблицы 2

Роль	Основные функции
Диспетчер	Просмотр всех заказов, назначение курьеров, управление маршрутными листами, формирование отчётов
Администратор	Управление пользователями, ведение справочников (зоны, тарифы, типы грузов), мониторинг системы

Каждая роль имеет собственное пространство экранов в приложении. Авторизация является обязательным условием для получения доступа к любой функции системы.

На основании анализа бизнес-процессов сформулированы требования к хранимым данным. Система должна оперировать следующими категориями сущностей:

- пользователи — учётные данные (логин, хэш пароля, роль), персональная информация, статус активности;
- клиенты и курьеры — расширенные профили, привязанные к базовой записи пользователя;
- зоны доставки — географические области обслуживания с указанием наименования и описания;
- типы грузов — категории отправок (документы, посылка, хрупкий груз и т.д.) с параметрами допустимого веса и объёма;
- тарифы — ставки оплаты за доставку в зависимости от зоны и типа груза;
- заказы — полная информация об отправлении: адреса, контакты, текущий статус, трек-номер, расчётная стоимость;
- сессии доставки — записи о фактическом выполнении доставки курьером с временными метками;
- история статусов — журнал всех изменений статуса заказа;

- подтверждения доставки — документирование факта передачи груза;
- уведомления — сообщения пользователям о событиях в системе;
- маршрутные листы — объединение заказов для совместного выполнения.

1.2 Разработка UML-диаграмм

Диаграмма вариантов использования (Use Case Diagram) описывает функциональные требования к системе с точки зрения взаимодействия акторов (пользователей) с системой¹. На рисунке 1 представлена диаграмма вариантов использования системы «FastDelivery».

¹ Ларман К. Применение UML и шаблонов проектирования. — 4-е изд. — М. : Вильямс, 2023. — 736 с.

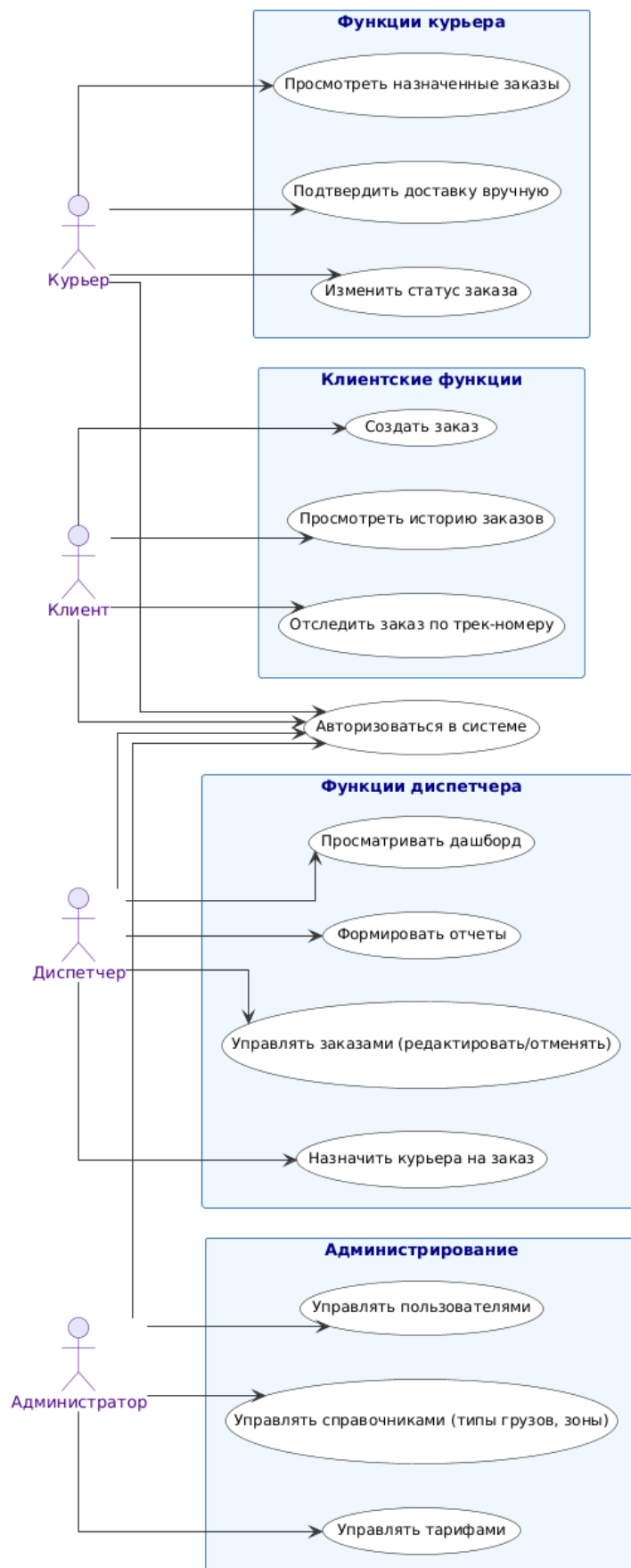


Рисунок 1 — Диаграмма вариантов использования (Use Case Diagram)

В диаграмме выделены четыре актора, соответствующие ролям пользователей: Клиент, Курьер, Диспетчер и Администратор. Все акторы без исключения взаимодействуют с вариантом использования «Авторизация» — вход в систему является обязательным условием доступа к любому функциональному блоку. После успешной авторизации система определяет роль пользователя и предоставляет доступ к соответствующему набору функций.

Для Клиента определены следующие варианты использования: «Создать заказ», «Просмотреть свои заказы», «Отследить статус заказа», «Отменить заказ», «Получить уведомление».

Курьер взаимодействует с вариантами: «Просмотреть назначенные заказы», «Принять заказ», «Изменить статус заказа», «Подтвердить доставку», «Просмотреть маршрутный лист».

Диспетчер имеет доступ к расширенному набору операций: «Просмотреть все заказы», «Назначить курьера», «Создать маршрутный лист», «Сформировать отчёт», «Отменить заказ».

Администратор реализует функции управления системой: «Управлять пользователями», «Управлять справочниками» (зоны, тарифы, типы грузов), «Мониторинг активности системы», а также унаследует все функции диспетчера через отношение <<include>>.

Группировка функций по ролям обеспечивает принцип наименьших привилегий: каждый пользователь видит только те данные и операции, которые необходимы для выполнения его должностных обязанностей.

Диаграмма последовательности (Sequence Diagram) описывает взаимодействие объектов системы во времени для конкретных сценариев. На рисунке 2 представлена диаграмма последовательности, охватывающая основные сценарии работы системы.

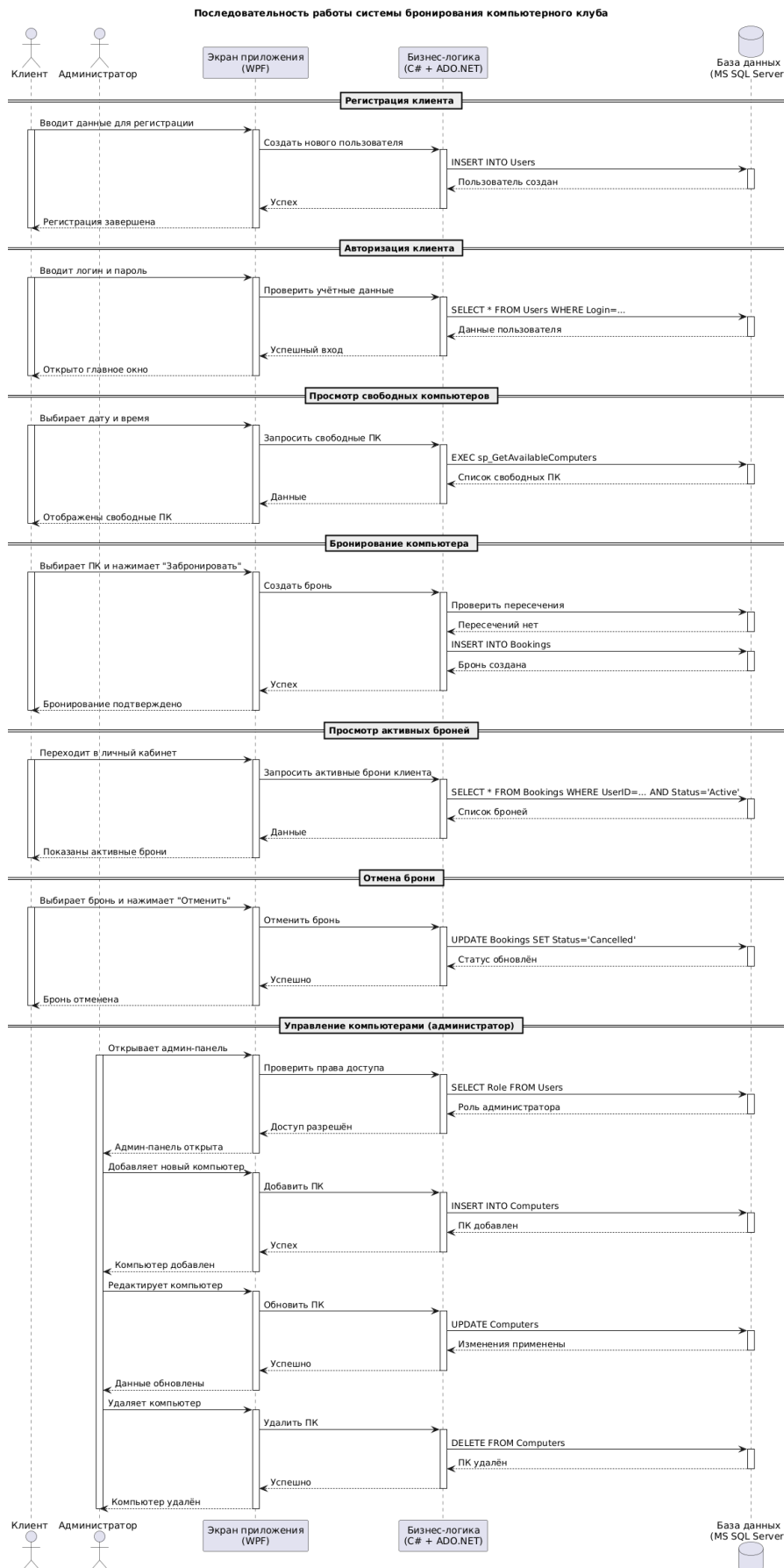


Рисунок 2 — Диаграмма последовательности (Sequence Diagram)

Диаграмма последовательности включает следующие основные потоки:

Сценарий 1: Создание заказа. Клиент через пользовательский интерфейс заполняет форму заказа и нажимает кнопку «Создать». Приложение передаёт запрос на уровень доступа к данным, который формирует запись в таблице Orders, генерирует уникальный трек-номер и фиксирует начальный статус New в таблице OrderStatusHistory. Система возвращает клиенту подтверждение с присвоенным трек-номером и создаёт уведомление.

Сценарий 2: Назначение курьера. Диспетчер открывает список заказов в статусе New и инициирует назначение. Приложение вызывает хранимую процедуру, которая анализирует свободных курьеров в зоне доставки заказа и выбирает подходящего. После успешного назначения статус заказа изменяется на Assigned, в таблицу OrderStatusHistory добавляется запись, курьер получает уведомление.

Сценарий 3: Работа курьера (изменение статусов и подтверждение). Курьер последовательно переводит заказ по цепочке статусов: Taken → InTransit → AtRecipient. При каждом переходе приложение обновляет запись в таблице Orders и добавляет строку в OrderStatusHistory. При подтверждении доставки (статус Delivered) перед записью срабатывает триггер, проверяющий наличие соответствующей записи в таблице DeliveryConfirmations. Если подтверждение отсутствует — операция блокируется, пользователь получает сообщение об ошибке.

Сценарий 4: Отслеживание заказа. Клиент вводит трек-номер. Приложение формирует запрос к базе данных и возвращает текущий статус заказа, а также историю всех его изменений из таблицы OrderStatusHistory.

Сценарий 5: Формирование отчёта. Диспетчер запрашивает отчёт об эффективности курьеров за период. Приложение обращается к представлению (VIEW), агрегирующему данные о количестве выполненных

доставок, среднем времени выполнения и прочих показателях. Результат отображается в виде таблицы или экспортируется.

1.3 Разработка макета приложения

Перед началом программной реализации были разработаны макеты ключевых экранов приложения, позволяющие согласовать логику интерфейса до написания кода.

Окно авторизации (рисунок 4) является точкой входа в систему для всех пользователей. Оно содержит два текстовых поля (логин и пароль), кнопку «Войти» и наименование системы «FastDelivery». Минималистичный дизайн обусловлен тем, что данный экран не несёт функциональной нагрузки — его единственная задача состоит в идентификации пользователя. После успешной проверки учётных данных приложение определяет роль пользователя и перенаправляет его на соответствующую главную панель.

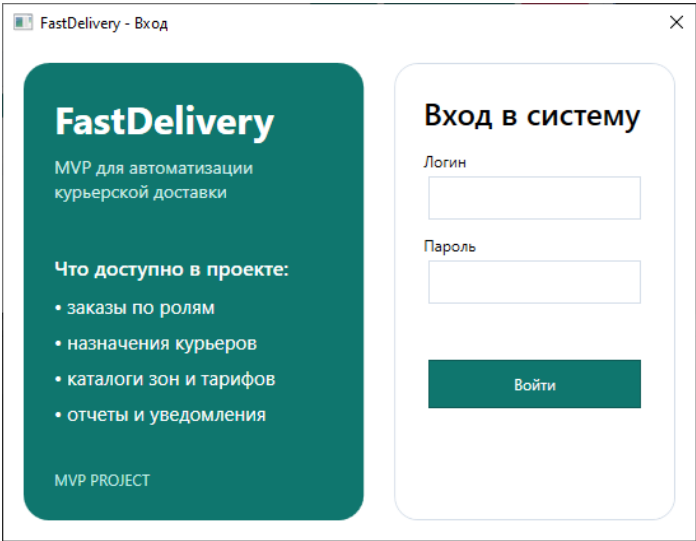


Рисунок 4 — Макет окна авторизации

Панель администратора (рисунок 5) построена по принципу вкладочной навигации. Горизонтальная панель вкладок содержит разделы: «Заказы», «Курьеры», «Справочники», «Отчёты», «Уведомления».

Основная рабочая область занята таблицей данных (DataGrid) с поддержкой сортировки и фильтрации. В нижней части предусмотрена панель действий с кнопками управления выбранной записью. Подобная структура экрана типична для бизнес-приложений: она обеспечивает быстрый доступ ко всем разделам без необходимости переключения между окнами.

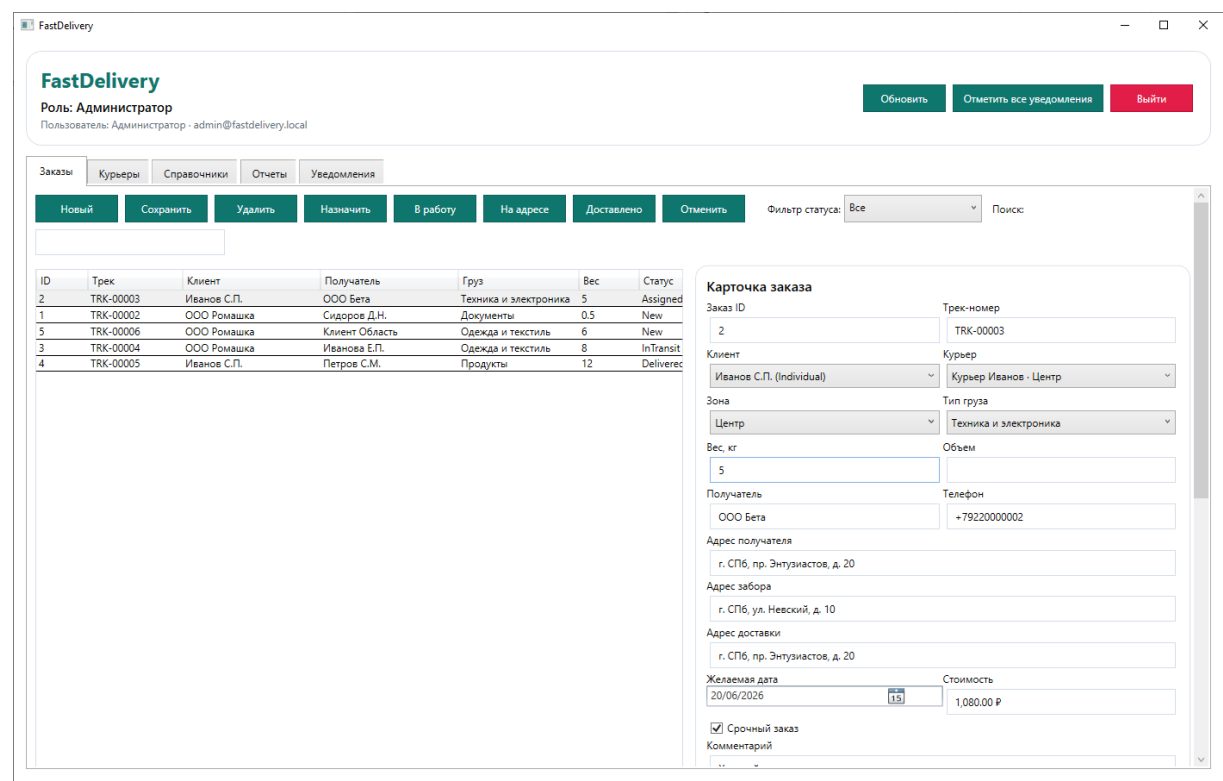


Рисунок 5 — Макет панели администратора

Выбор вкладочного интерфейса обусловлен несколькими соображениями: все функции системы остаются в одном окне, что снижает когнитивную нагрузку на пользователя; навигация реализована единообразно для всех ролей (различается лишь набор видимых вкладок); расположение таблицы данных в центре экрана отражает ключевую сущность — список заказов или объектов справочника.

2 Проектирование и разработка базы данных

2.1 Разработка ER-диаграммы на основе сырых данных

На основании анализа предметной области разработана концептуальная модель данных в виде ER-диаграммы (Entity-Relationship Diagram).

На рисунке 3 представлена ER-диаграмма системы «FastDelivery»².

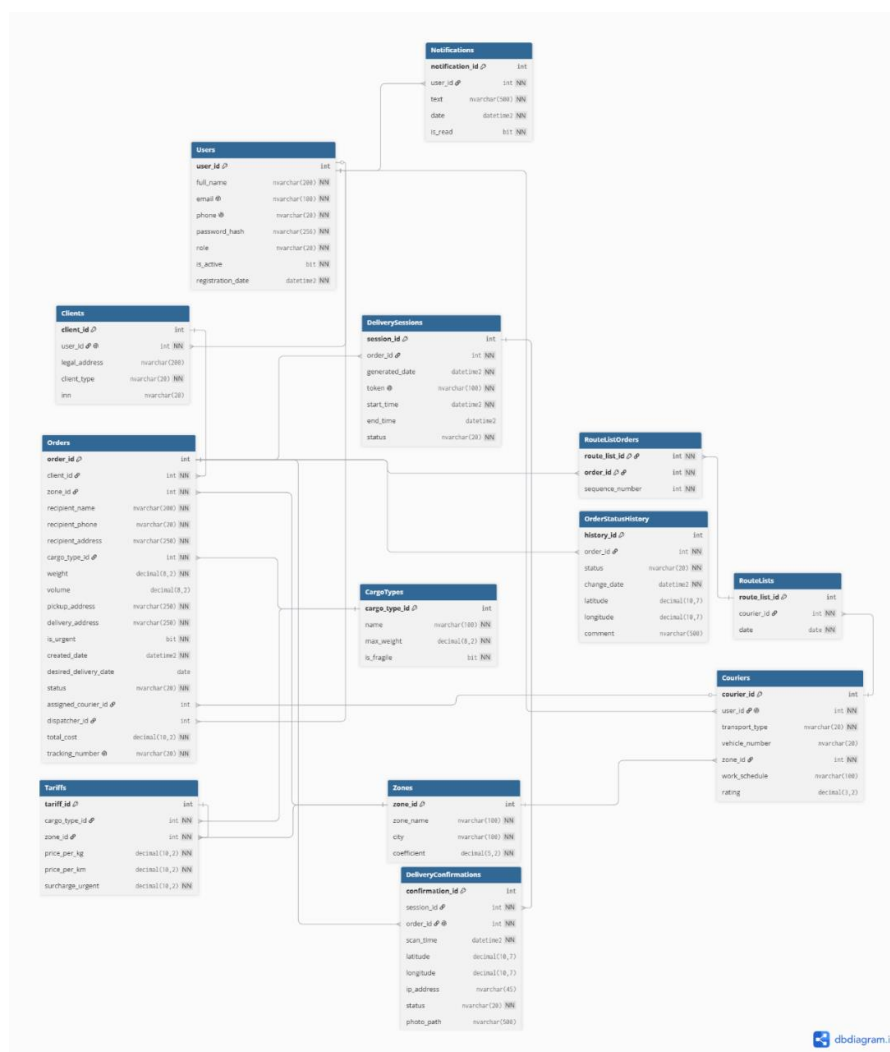


Рисунок 3 — ER-диаграмма базы данных системы «FastDelivery»

² Карпова Т. С. Базы данных: модели, разработка, реализация. — СПб. : Питер, 2022. — 304 с.

Модель включает тринадцать сущностей, перечень которых приведён в таблице 3.

Таблица 3 — Сущности базы данных

Сущность	Назначение
Users	Базовые учётные данные всех пользователей системы
Clients	Расширенный профиль клиента
Couriers	Расширенный профиль курьера
Zones	Справочник зон доставки
CargoTypes	Справочник типов грузов
Tariffs	Тарифы для сочетания зоны и типа груза
Orders	Заказы на доставку
DeliverySessions	Сессии выполнения доставки курьером
OrderStatusHistory	Журнал изменений статусов заказов
DeliveryConfirmations	Подтверждения факта доставки
Notifications	Уведомления пользователей
RouteLists	Маршрутные листы
RouteListOrders	Связующая таблица маршрутных листов и заказов

Сущность Users является базовой для всей системы пользователей. Она хранит логин, хэш пароля, имя, фамилию, контактный телефон, адрес электронной почты и роль (перечисляемый тип). Сущности Clients и Couriers реализуют паттерн «наследование таблицами»: каждая из них содержит внешний ключ, ссылающийся на Users, и дополнительные

атрибуты, специфичные для роли. Для клиента это — адрес доставки по умолчанию; для курьера — текущая зона работы и признак доступности (IsAvailable).

Центральной сущностью модели является Orders. Она агрегирует все параметры заказа: ссылки на клиента и назначенного курьера, тип груза, зону доставки, адреса отправки и получения, контактные данные получателя, трек-номер, текущий статус, расчётную стоимость, а также временные метки создания и планируемого срока доставки.

Сущность Tariffs описывает ценовую политику: для каждой пары «зона доставки — тип груза» задаётся базовая ставка. Это позволяет гибко управлять ценообразованием без изменения логики приложения.

DeliverySessions фиксируют факт выполнения доставки: момент начала, завершения, количество километров пробега. OrderStatusHistory ведёт полный аудит-журнал переходов статусов с указанием инициатора изменения и комментария.

RouteLists и связующая таблица RouteListOrders реализуют концепцию маршрутного листа: один лист содержит несколько заказов, назначенных одному курьеру на один рейс. Это отношение «многие-ко-многим» нормализовано через промежуточную таблицу с дополнительным атрибутом — порядком следования (SortOrder).

В модели преобладают связи типа «один-ко-многим»: один пользователь — один профиль клиента или курьера; одна зона — много заказов; один клиент — много заказов; один заказ — много записей в истории статусов. Единственная связь «многие-ко-многим» — между RouteLists и Orders — разрешена через таблицу RouteListOrders.

В качестве первичных ключей во всех таблицах используются суррогатные целочисленные идентификаторы (INT IDENTITY). Данное решение обосновывается следующими доводами:

- независимость от предметной области. Трек-номер заказа является бизнес-ключом и может быть изменён или сгенерирован повторно; суррогатный ключ остаётся неизменным на всё время жизни записи;
- производительность. Целочисленные ключи обеспечивают более быстрое объединение таблиц по сравнению с составными или строковыми ключами;
- простота внешних ключей. Все ссылки между таблицами унифицированы: ссылающийся столбец всегда имеет тип INT.

2.2 Разработка физической модели базы данных в MS SQL Server

Физическая модель реализована в системе управления базами данных Microsoft SQL Server³. При выборе типов данных применялись следующие принципы:

- для хранения текстовых данных переменной длины используется тип NVARCHAR с ограничением максимального размера — это обеспечивает поддержку Unicode при минимальном расходе дискового пространства;
- денежные суммы (стоимость доставки, тарифные ставки) хранятся в типе DECIMAL(10,2), исключающем ошибки округления, характерные для типов с плавающей запятой;
- временные метки записываются как DATETIME2, обеспечивающий большую точность и расширенный диапазон дат по сравнению с устаревшим типом DATETIME;
- логические признаки (IsAvailable, IsRead и т.п.) представлены типом BIT;

³ Петкович Д. Microsoft SQL Server 2019. Введение в T-SQL. — СПб. : БХВ-Петербург, 2021. — 496 с.

- статусы заказов, а также роли пользователей хранятся в виде строковых значений типа NVARCHAR с ограничениями CHECK, перечисляющими допустимые значения.

Во всех таблицах применён набор стандартных ограничений целостности. Ограничение NOT NULL установлено для всех обязательных атрибутов. Ограничение UNIQUE применяется к столбцам, значения которых должны быть уникальными в пределах таблицы: логин пользователя (поле Login в таблице Users), трек-номер заказа (поле TrackingNumber в таблице Orders). Ограничения CHECK ограничивают множество допустимых значений для статусных и перечислимых полей: например, статус заказа может принимать только одно из семи предусмотренных значений.

Для ускорения наиболее частых запросов в базе данных созданы некластерные индексы по следующим столбцам:

- TrackingNumber в таблице Orders — поиск заказа по трек-номеру является наиболее распространённой операцией при отслеживании доставки;

- Status в таблице Orders — фильтрация заказов по текущему статусу используется в списках диспетчера и курьера;

- CreatedAt в таблице Orders — сортировка и фильтрация по дате создания заказа применяется при формировании отчётов;

- OrderId в таблице OrderStatusHistory — быстрый поиск истории конкретного заказа;

- UserId в таблицах Clients и Couriers — поиск профиля по идентификатору базовой записи пользователя;

- CourierId в таблице Orders — выборка заказов, назначенных конкретному курьеру.

Выбор перечисленных столбцов для индексирования обусловлен анализом типовых запросов системы: именно они фигурируют в условиях

фильтрации (WHERE) и объединения (JOIN) в наиболее нагруженных операциях.

Поведение внешних ключей при изменении и удалении родительской записи определяется стратегией каскадирования. В базе данных применяются три стратегии.

ON DELETE CASCADE используется для дочерних записей, которые лишены смысла без родительской: записи в OrderStatusHistory удаляются вместе с удалением заказа; записи в RouteListOrders удаляются вместе с маршрутным листом.

ON DELETE SET NULL применяется в ситуациях, когда дочерняя запись сохраняет смысл и без родительской, но связь разрывается: внешний ключ FK_Orders_Couriers при удалении курьера устанавливает значение NULL в поле CourierId таблицы Orders — заказ не удаляется, а лишь теряет привязку к конкретному курьеру, что позволяет диспетчеру назначить другого исполнителя.

NO ACTION (RESTRICT) применяется там, где удаление родительской записи при наличии дочерних недопустимо: например, нельзя удалить пользователя, у которого есть активные заказы или профиль клиента/курьера.

Триггер trg_OrderStatusHistory_CheckDelivery срабатывает при добавлении новой записи в таблицу OrderStatusHistory со статусом Delivered. Логика триггера такова: перед фиксацией данного статуса выполняется проверка наличия соответствующей записи в таблице DeliveryConfirmations для данного заказа⁴. Если подтверждение доставки ещё не создано, триггер откатывает транзакцию и генерирует информативное сообщение об ошибке. Это гарантирует, что ни одному заказу не может быть присвоен статус «Доставлен» без документирования

⁴ Microsoft. Документация по SQL Server — Transact-SQL [Электронный ресурс] // Microsoft Learn. — URL: <https://learn.microsoft.com/ru-ru/sql/t-sql/language-reference> (дата обращения: 10.05.2026).

факта передачи груза получателю — вне зависимости от того, каким способом приложение производит запись в базу данных.

Хранимая процедура `usp_AssignCourier` автоматизирует выбор и назначение курьера для указанного заказа. Процедура принимает идентификатор заказа в качестве входного параметра. Её логика включает следующие шаги:

- определить зону доставки, к которой относится заказ;
- выбрать из таблицы `Couriers` свободного курьера (`IsAvailable = 1`), работающего в данной зоне, с наименьшим числом текущих активных заказов.
- обновить поле `CourierId` в таблице `Orders`;
- установить флаг занятости курьера (`IsAvailable = 0`);
- добавить запись о смене статуса заказа на `Assigned` в таблицу `OrderStatusHistory`;
- создать уведомление для курьера.

Процедура выполняется в рамках транзакции: если на любом из шагов возникает ошибка (например, не найдено доступных курьеров), вся последовательность операций откатывается, обеспечивая согласованность данных.

Функция `fn_CalculateDeliveryCost` вычисляет стоимость доставки для заказа с учётом тарифа. Входными параметрами являются идентификатор зоны и идентификатор типа груза. Функция обращается к таблице `Tariffs`, извлекает базовую ставку и применяет к ней коэффициенты — например, надбавку за срочность или за вес груза, превышающий пороговое значение. Использование функции позволяет централизовать логику ценообразования: при изменении тарифной политики достаточно изменить одну функцию, а не перепрограммировать несколько участков приложения.

Представление vw_CourierEfficiency формирует агрегированную статистику по каждому курьеру: общее число выполненных доставок, количество отмен, средний фактический срок доставки, доля заказов, выполненных в срок. Данные извлекаются объединением таблиц Couriers, Orders и DeliverySessions. Представление используется диспетчером для анализа качества работы и отображается в соответствующем разделе вкладки «Отчёты».

Скрипт создания базы данных (включающий все таблицы, индексы, ограничения, триггер, процедуру и функцию) размещён в репозитории GitHub по адресу <https://github.com/MG178/vlasov> в файле database/schema.sql.

2.3 Разработка и именование связей и ограничений

Единообразное именование объектов базы данных является важным аспектом поддерживаемости системы. В проекте приняты следующие соглашения:

- внешние ключи именуются по шаблону FK_ДочерняяТаблица_РодительскаяТаблица. Например, FK_Orders_Clients обозначает внешний ключ в таблице Orders, ссылающийся на таблицу Clients; FK_Orders_Couriers — ссылку Orders → Couriers; FK_OrderStatusHistory_Orders — связь таблицы истории статусов с заказом.

- первичные ключи именуются PK_ИмяТаблицы;
 - индексы именуются по шаблону IX_ИмяТаблицы_ИмяСтолбца;
 - ограничения CHECK именуются CHK_ИмяТаблицы_ИмяСтолбца;
 - ограничения UNIQUE именуются UQ_ИмяТаблицы_ИмяСтолбца;
 - хранимые процедуры имеют префикс usp_; функции — fn_;
- триггеры — trg_; представления — vw_.

В таблице 4 приведены основные внешние ключи базы данных с описанием их смысла и поведения при удалении родительской записи.

Таблица 4 — Ключевые внешние ключи базы данных

Имя ключа	Дочерняя таблица	Родительская таблица	Поведение при удалении
FK_Clients_Users	Clients	Users	NO ACTION
FK_Couriers_Users	Couriers	Users	NO ACTION
FK_Couriers_Zones	Couriers	Zones	SET NULL
FK_Orders_Clients	Orders	Clients	NO ACTION
FK_Orders_Couriers	Orders	Couriers	SET NULL
FK_Orders_Zones	Orders	Zones	NO ACTION
FK_Orders_CargoTypes	Orders	CargoTypes	NO ACTION
FK_OrderStatusHistory_Orders	OrderStatusHistory	Orders	CASCADE
FK_DeliveryConfirmations_Orders	DeliveryConfirmations	Orders	CASCADE

Продолжение таблицы 4

Имя ключа	Дочерняя таблица	Родительская таблица	Поведение при удалении
FK_DeliverySessions_Orders	DeliverySessions	Orders	CASCADE
FK_Notifications_Users	Notifications	Users	CASCADE
FK_RouteListOrders_RouteLists	RouteListOrders	RouteLists	CASCADE
FK_RouteListOrders_Orders	RouteListOrders	Orders	CASCADE
FK_Tariffs_Zones	Tariffs	Zones	NO ACTION
FK_Tariffs_CargoTypes	Tariffs	CargoTypes	NO ACTION

Целостность данных в реляционной базе данных обеспечивается комплексом механизмов, которые действуют независимо от уровня приложения. Это принципиально важно: при прямых операциях с базой (через среду администрирования или сторонний инструмент) ограничения по-прежнему будут работать.

Ссылочная целостность, поддерживаемая внешними ключами, исключает появление «висячих» ссылок — записей, ссылающихся на несуществующих родителей. Например, невозможно создать запись о смене статуса для заказа, которого нет в системе.

Доменная целостность, поддерживаемая ограничениями CHECK и типами данных, гарантирует, что значения в столбцах соответствуют

допустимым диапазонам. Так, статус заказа не может принять произвольное строковое значение — только одно из семи определённых.

Целостность сущности, поддерживаемая первичными ключами и ограничениями UNIQUE, исключает дублирование записей. Два заказа не могут иметь одинаковый трек-номер; два пользователя — одинаковый логин.

Совокупность перечисленных механизмов формирует надёжный «контракт» базы данных, который гарантирует корректность хранимых данных при любом способе взаимодействия с системой.

3 Проектирование и разработка приложения

3.1 Разработка прототипа приложения

Клиентское приложение «FastDelivery» разработано с использованием технологии Windows Presentation Foundation (WPF) на языке программирования C#⁵. WPF является частью платформы .NET и предоставляет развитые средства для построения настольных приложений с богатым пользовательским интерфейсом: поддержку стилей, привязки данных (data binding), анимаций и векторной графики.

Архитектурно приложение организовано по принципу разделения ответственности: логика работы с данными отделена от логики пользовательского интерфейса. Пользовательский интерфейс описывается в файлах разметки XAML, тогда как обработка событий, обращения к данным и бизнес-логика реализованы в классах-обработчиках (code-behind). Хранение данных в текущей учебной MVP-версии реализовано посредством коллекций объектов в оперативной памяти, однако архитектура приложения предусматривает полноценный переход на MS SQL Server: классы доступа к данным инкапсулированы в отдельном слое и могут быть заменены на реализации с использованием ADO.NET или Entity Framework Core без изменения логики интерфейса.

После запуска приложения пользователю отображается окно авторизации. Введённые учётные данные проверяются по хранилищу пользователей. При успешной аутентификации загружается главное окно MainWindow, конфигурация которого определяется ролью вошедшего пользователя: набор видимых вкладок и доступных операций формируется динамически.

⁵ Microsoft. Документация по WPF — Windows Presentation Foundation [Электронный ресурс] // Microsoft Learn. — URL: <https://learn.microsoft.com/ru-ru/dotnet/desktop/wpf/>

Для демонстрации и тестирования системы предусмотрены следующие демо-аккаунты, перечисленные в таблице 5.

Таблица 5 — Демонстрационные учётные записи

Логин	Пароль	Роль	Доступные функции
admin	admin	Администратор	Все разделы системы
dispatcher	dispatcher	Диспетчер	Заказы, Курьеры, Отчёты, Уведомления
courier1	courier1	Курьер	Назначенные заказы, смена статусов
client1	client1	Клиент	Создание и просмотр своих заказов

Вкладка «Заказы» отображает список заказов в виде таблицы (компонент DataGrid). Набор отображаемых записей фильтруется по роли: диспетчер и администратор видят все заказы; клиент — только свои; курьер — только назначенные ему. Для каждой строки доступны операции в зависимости от роли и текущего статуса заказа. При двойном щелчке по строке открывается форма с детальной информацией о заказе: полными адресами, историей статусов, контактами.

Вкладка «Курьеры» (доступна диспетчеру и администратору) содержит список курьеров с указанием их текущего статуса (доступен / занят) и зоны работы. Из этой вкладки диспетчер может вручную назначить курьера на заказ или запустить автоматическое назначение через соответствующую процедуру.

Вкладка «Справочники» (доступна только администратору) предоставляет интерфейс для управления нормативно-справочной информацией: зонами доставки, типами грузов и тарифами. Каждый

справочник отображается в отдельной вложенной вкладке с поддержкой операций добавления, редактирования и удаления записей.

Вкладка «Отчёты» позволяет диспетчеру и администратору формировать аналитические выборки. Доступны следующие отчёты: количество заказов по статусам, эффективность курьеров за период, динамика числа заказов по дням. Данные отображаются в табличном виде; функциональность экспорта предусмотрена как направление дальнейшего развития.

Вкладка «Уведомления» отображает список уведомлений, адресованных текущему пользователю, с разделением на прочитанные и непрочитанные. Уведомления генерируются автоматически при ключевых событиях: назначении курьера, смене статуса заказа, подтверждении доставки.

В проектируемой архитектуре обращения к базе данных сосредоточены в классах слоя доступа к данным (Data Access Layer). Каждому логическому разделу системы соответствует отдельный класс-репозиторий: OrderRepository, CourierRepository, UserRepository и т.д. Репозиторий инкапсулирует запросы к базе и возвращает объекты доменной модели.

Для выполнения операций, затрагивающих несколько таблиц (назначение курьера, подтверждение доставки), приложение вызывает хранимые процедуры — это обеспечивает атомарность операций и перекладывает контроль транзакций на сторону сервера баз данных. Простые операции чтения (списки заказов, справочники) выполняются через представления или параметрические запросы. Расчёт стоимости доставки при создании нового заказа производится вызовом скалярной функции fn_CalculateDeliveryCost.

3.2 Разработка описания проекта в GitHub

Исходный код проекта размещён в публичном репозитории GitHub по адресу: <https://github.com/MG178/vlasov>⁶. На рисунках 6 и 7 представлены скриншоты главной страницы репозитория и отражающий его структуру.



Рисунок 6 — Главной страницы репозитория GitHub

⁶ Власов В. А. FastDelivery: клиентское приложение для автоматизации логистики курьерской доставки [Электронный ресурс] // GitHub. — URL: <https://github.com/MG178/vlasov> (дата обращения: 18.06.2026).

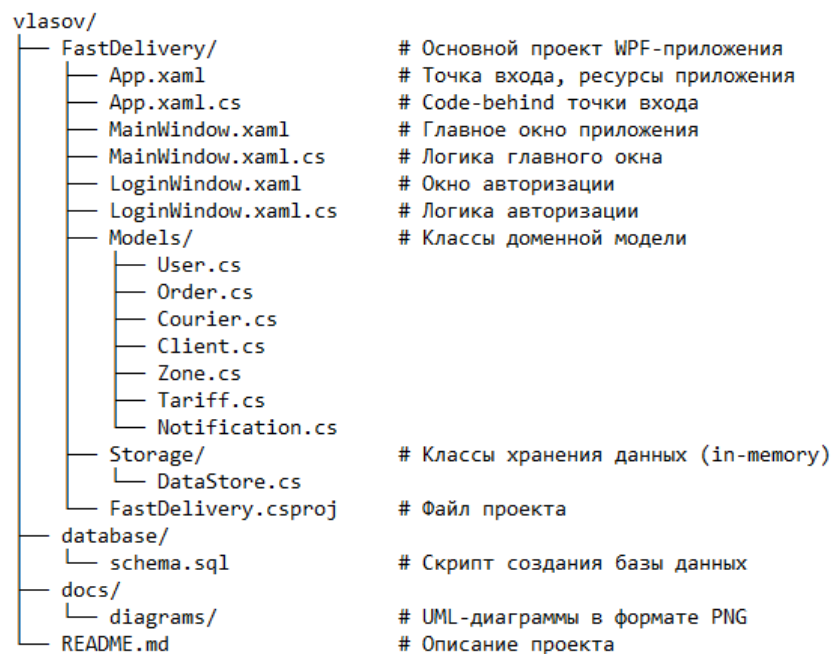


Рисунок 7 — Структура репозитория GitHub

App.xaml является точкой входа в WPF-приложение. В нём объявляются глобальные ресурсы: словари стилей, цветовые схемы, шаблоны элементов управления, используемые во всех окнах приложения.

MainWindow.xaml содержит разметку главного окна с вкладочной навигацией. В его code-behind реализована логика инициализации: после авторизации в MainWindow передаётся объект текущего пользователя, на основании роли которого скрываются или отображаются соответствующие вкладки.

LoginWindow.xaml реализует форму авторизации. В code-behind выполняется поиск пользователя по логину в хранилище DataStore и сравнение введённого пароля.

Папка Models/ содержит РОСО-классы (Plain Old CLR Object), отражающие сущности предметной области. Каждый класс содержит свойства, соответствующие столбцам соответствующей таблицы базы данных. Например, класс Order содержит свойства OrderId, TrackingNumber, Status, ClientId, CourierId, PickupAddress, DeliveryAddress, CalculatedCost и т.д.

Класс DataStore в папке Storage реализует паттерн «хранилище в памяти» для MVP-версии. Он содержит статические коллекции (List<T>) для каждой сущности и предоставляет методы доступа к ним. При запуске приложения DataStore заполняется тестовыми данными, включая демонстрационные аккаунты.

schema.sql в папке database — полный скрипт создания базы данных MS SQL Server, включающий определения всех таблиц, индексов, внешних ключей, ограничений, а также триггер trg_OrderStatusHistory_CheckDelivery, хранимую процедуру usp_AssignCourier, функцию fn_CalculateDeliveryCost и представление vw_CourierEfficiency.

Файл README.md на главной странице репозитория содержит следующие разделы:

- краткое описание проекта — тема, цель, используемые технологии;
- требования — версия .NET, Visual Studio, MS SQL Server (для полной версии);
- инструкция по установке и запуску — клонирование репозитория, открытие решения в Visual Studio, запуск в режиме отладки;
- демо-аккаунты — таблица с логинами, паролями и ролями;
- структура проекта — краткое описание папок;

Наличие подробного README обеспечивает возможность быстрого запуска и изучения проекта любым разработчиком, получившим доступ к репозиторию, без необходимости изучать исходный код.

Заключение

В ходе выполнения данного курсового проекта были реализованы все поставленные задачи.

Проведён анализ предметной области — системы курьерской доставки малогабаритных грузов. Выявлены ключевые бизнес-процессы: создание и обработка заказа, назначение курьера, отслеживание статусов, подтверждение доставки, формирование отчётности. Определены роли пользователей системы (Клиент, Курьер, Диспетчер, Администратор) и требования к хранимым данным.

Разработаны UML-диаграммы: диаграмма вариантов использования, описывающая функциональные требования в разбивке по ролям, и диаграмма последовательности, формализующая основные сценарии взаимодействия пользователей с системой. Разработаны макеты ключевых экранов приложения — окна авторизации и панели администратора.

Спроектирована база данных: разработана концептуальная ER-диаграмма, включающая тринадцать сущностей с обоснованными связями и суррогатными ключами; реализована физическая модель в MS SQL Server с применением ограничений целостности, индексов для оптимизации запросов и каскадных действий. Созданы объекты серверной логики: триггер проверки факта доставки, хранимая процедура автоматического назначения курьера, функция расчёта стоимости и представление эффективности курьеров.

Разработано клиентское WPF-приложение «FastDelivery», реализующее ролевой доступ и охватывающее основные функциональные блоки: управление заказами, работу с курьерами, ведение справочников, уведомления и отчётность. Исходный код размещён в публичном репозитории GitHub с подробной документацией.

Таким образом, цель проекта достигнута: разработана информационная система, автоматизирующая основные логистические

процессы курьерской доставки и обеспечивающая структурированное хранение данных, разграничение прав доступа и прозрачность бизнес-процессов для всех участников.

В качестве направлений дальнейшего развития системы можно выделить:

- переход на MS SQL Server — замена in-memory хранилища на полноценную СУБД с использованием созданного скрипта schema.sql;
- интеграция с картографическими сервисами — визуализация маршрутов курьеров на карте, геокодирование адресов;
- мобильное приложение для курьеров — нативный клиент на Android/iOS с push-уведомлениями и GPS-трекингом;
- QR-коды для подтверждения доставки — генерация уникальных кодов для заказов, сканирование при вручении получателю;
- система аналитики и дашбордов — расширенная отчётность с визуализацией KPI в реальном времени;
- REST API — выделение серверной части в отдельный сервис для обеспечения работы нескольких клиентов.

					АТТ2. КП0326. 000 ПЗ	Лист
						34
Изм.	Лист	№ докум.	Подп.	Дата		

Литература

Власов В. А. FastDelivery: клиентское приложение для автоматизации логистики курьерской доставки [Электронный ресурс] // GitHub. — URL: <https://github.com/MG178/vlasov> (дата обращения: 18.06.2026).

Голицына О. Л., Максимов Н. В., Партыка Т. Л. Базы данных. — М. : Форум, 2022. — 448 с.

Карпова Т. С. Базы данных: модели, разработка, реализация. — СПб. : Питер, 2022. — 304 с.

Ларман К. Применение UML и шаблонов проектирования. — 4-е изд. — М. : Вильямс, 2023. — 736 с.

Петкович Д. Microsoft SQL Server 2019. Введение в T-SQL. — СПб. : БХВ-Петербург, 2021. — 496 с. — ISBN 978-5-9775-6694-9.

Троелсен Э., Джепикс Ф. Язык программирования C# 10 и платформа .NET 6. — М. : Вильямс, 2022. — 1376 с.

Microsoft. Документация по SQL Server — Transact-SQL [Электронный ресурс] // Microsoft Learn. — URL: <https://learn.microsoft.com/ru-ru/sql/t-sql/language-reference> (дата обращения: 10.05.2026).

Microsoft. Документация по WPF — Windows Presentation Foundation [Электронный ресурс] // Microsoft Learn. — URL: <https://learn.microsoft.com/ru-ru/dotnet/desktop/wpf/> (дата обращения: 10.05.2026).

Stack Overflow. Вопросы и ответы по разработке на CWPf и MS SQL Server [Электронный ресурс]. — URL: <https://stackoverflow.com/questions/tagged/c%23+wpf+sql-server> (дата обращения: 12.05.2026).